

Anthony Pham
Dr. Eman Ramadan
CSCI 4211
14 October 2025

Project 1 Report

Implementing Multiple Client Functionality

To have the server be able to handle multiple clients simultaneously, I opted to use the *threading* library. This library provides the *Thread* class, which allowed me to assign each client socket to an independent thread running the `trivia_game()` function. Having multiple threads running concurrently meant that I needed to be mindful of how each thread accesses the shared resources on my laptop or server. Most importantly, I needed to ensure that only one thread accesses the `client_socket` set at a time when adding or removing from the set. This is where the `Lock` class comes in. When a `connection_socket` (or `client_socket`) is added, discarded, shut down, or closed, a `Lock` object is there to make sure that only one thread is accessing the set at a time.

An additional functionality I implemented was a timeout for each client, where the server will close the connection socket if nothing is heard from the client for a minute. This is to prevent the set of client sockets from getting cluttered with useless connections.

Deploying the Server

I was successful in deploying the server in the cloud through AWS's EC2 cloud service. The server has a public IP address of 18.191.255.56 and uses port 5001.

us-east-2.console.aws.amazon.com

Search [Option+S]

United States (Ohio) Account ID: 5133-7132-2325 pham0556

```
[SERVER]: Connection with ('131.212.248.172', 39085) has closed.
[SERVER]: Connected by ('131.212.248.172', 39133)
[SERVER]: Waiting for client ('131.212.248.172', 39133) to start a new game...
[SERVER]: Client ('131.212.248.172', 39133) has started a new game!
[SERVER]: Sent question 1 to ('131.212.248.172', 39133). The correct answer is 1.
[FROM ('131.212.248.172', 39133)]: 1
[TO ('131.212.248.172', 39133)]: That is correct! You scored 1 point.

[SERVER]: Sent question 2 to ('131.212.248.172', 39153). The correct answer is 1.
[FROM ('131.212.248.172', 39153)]: 3
[TO ('131.212.248.172', 39153)]: That is correct! You scored 1 point.

[SERVER]: Sent question 3 to ('131.212.248.172', 39153). The correct answer is 1.
[FROM ('131.212.248.172', 39153)]: 1
[TO ('131.212.248.172', 39153)]: That is correct! You scored 1 point.

[SERVER]: Sent question 4 to ('131.212.248.172', 39153). The correct answer is 4.
ubuntu@ip-172-31-44-196:~/CSCI4211-Project1$ tail -n 20 server.log

[SERVER]: Error with ('131.212.248.172', 39085) -> [Errno 32] Broken pipe
[SERVER]: Connection with ('131.212.248.172', 39085) has closed.

[SERVER]: Connected by ('131.212.248.172', 39133)
[SERVER]: Waiting for client ('131.212.248.172', 39133) to start a new game...
[SERVER]: Client ('131.212.248.172', 39133) has started a new game!
[SERVER]: Sent question 1 to ('131.212.248.172', 39133). The correct answer is 1.
[FROM ('131.212.248.172', 39133)]: 1
[TO ('131.212.248.172', 39133)]: That is correct! You scored 1 point.

[SERVER]: Sent question 2 to ('131.212.248.172', 39153). The correct answer is 1.
[FROM ('131.212.248.172', 39153)]: 3
[TO ('131.212.248.172', 39153)]: That is correct! You scored 1 point.

[SERVER]: Sent question 3 to ('131.212.248.172', 39153). The correct answer is 1.
[FROM ('131.212.248.172', 39153)]: 1
[TO ('131.212.248.172', 39153)]: That is correct! You scored 1 point.

[SERVER]: Sent question 4 to ('131.212.248.172', 39153). The correct answer is 4.
ubuntu@ip-172-31-44-196:~/CSCI4211-Project1$
```

i-0549bf4312bb7cd7e (CSCI 4211 Project 1)

PublicPs: 18.191.255.56 PrivatePs: 172.31.44.196

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Terminal output is only printed in the server's deployed code for debug purposes.

us-east-2.console.aws.amazon.com

Search [Option+S]

United States (Ohio) Account ID: 5133-7132-2325 pham0556

EC2 > Instances > i-0549bf4312bb7cd7e

EC2

- Dashboard
- EC2 Global View
- Events
- ▼ Instances
 - Instances
 - Instance Types
 - Launch Templates
 - Spot Requests
 - Savings Plans
 - Reserved Instances
 - Dedicated Hosts
 - Capacity Reservations
- ▼ Images
 - AMIs
 - AMI Catalog
- ▼ Elastic Block Store
 - Volumes
 - Snapshots
 - Lifecycle Manager
- ▼ Network & Security
 - Security Groups
 - Elastic IPs
 - Placement Groups
 - Key Pairs
 - Network Interfaces

Instance summary for i-0549bf4312bb7cd7e (CSCI 4211 Project 1)

Updated less than a minute ago

Instance ID i-0549bf4312bb7cd7e	Public IPv4 address 18.191.255.56 open address	Private IPv4 addresses 172.31.44.196
IPv6 address -	Instance state Running	Public DNS ec2-18-191-255-56.us-east-2.compute.amazonaws.com open address
Hostname type IP name: ip-172-31-44-196.us-east-2.compute.internal	Private IP DNS name (IPv4 only) ip-172-31-44-196.us-east-2.compute.internal	Elastic IP addresses -
Answer private resource DNS name IPv4 (A)	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendation s. Learn more
Auto-assigned IP address 18.191.255.56 [Public IP]	VPC ID vpc-0817ffce80282b64	Auto Scaling Group name -
IAM Role -	Subnet ID subnet-02f45e037e9aa7f0e	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:us-east-2:513371322325:instance/i-0549bf4312bb7cd7e	
Operator -		

Details Status and alarms Monitoring Security Networking Storage Tags

▼ Instance details

Cloud Setup Experience

Navigating AWS and the initial server setup was the biggest challenge for me, as I had never deployed a server nor used AWS. AWS offers a *large* number of services, so it was difficult and daunting to decide which service to use to deploy my code. After some research, I decided to go with EC2 as it has plenty of online resources, is accessible to me as a free user, and seemed to fit my needs the most without any extra fluff while also giving me control over configuring the server.

I found it a bit straightforward to create the instance, and it helped that the AWS website provides you with a web terminal to connect to the server. I was then given a public IPv4 address to plug into the client code and set the server up to use port 5001 for connecting. Then I uploaded my code to a GitHub repo from my laptop, where I cloned the code from the server and ran the Python server code. But that's where I ran into my second issue: after a while, the SSH connection to the server would timeout and disconnect, which meant that the server code would stop running.

Earlier, I had assumed that the server setup was complete, but when I attempted to connect to the server again after a couple of hours, the client was unable to connect to the server. That's when I learned that the SSH session isn't persistent. The solution was to use the *nohup* command, which ignores the hangup signal when the SSH disconnects and keeps the server code running in the background. I had the server output go to a *server.log* file so that I can access the output at a later time. After trying this solution, the server seems to be persistently running when I tried connecting to the server at several later attempts.